

Relatório de integração com MOM — Sprint 2

Objetivo

Integrar um middleware orientado a mensagens (MOM) ao backend do FestaLink e usar essa camada para tornar a notificação do prestador assíncrona, atendendo ao item 2.2 do enunciado, que exige arquitetura orientada a eventos e veda polling contínuo no app do prestador.

A meta acrescentada na execução foi que a notificação seja durável: se o processo que escuta a fila estiver fora do ar quando o evento for publicado, a mensagem precisa ser entregue assim que ele reconectar, sem perda.

Tecnologias escolhidas

A escolha foi RabbitMQ provisionado via Docker Compose, em container dedicado ao projeto (`festalink-rabbitmq`, porta 5672 do host para AMQP e 15672 para a UI de administração). RabbitMQ oferece filas duráveis com mensagens persistentes em disco, o que atende diretamente o requisito de durabilidade. A primeira tentativa foi com Redis Pub/Sub, mas o modo pub/sub do Redis é fire-and-forget por design e descarta a mensagem se nenhum subscriber estiver conectado no instante da publicação. Migrar para RabbitMQ foi a forma de resolver esse problema sem fugir das opções permitidas pelo enunciado.

A topologia escolhida no RabbitMQ é um exchange `topic` durável (`festalink.eventos`) com uma fila durável (`festalink.worker`) ligada por routing key `reserva.#`. As mensagens são publicadas com `persistent: true`, e o consumer faz `prefetch(1)` com `ack manual` e `nack` com `requeue` em caso de erro. A biblioteca usada no lado Node é o `amqplib`.

Para empurrar o evento ao app do prestador foi adotada a biblioteca `ws`, que oferece um WebSocket server mínimo, sem protocolo proprietário em cima. A autenticação no handshake reaproveita o `TokenService` já usado pelo middleware HTTP, mantendo uma única fonte de verdade para JWT.

Arquitetura final

O backend foi dividido em dois processos independentes, alinhando o desenho à exigência de comunicação assíncrona via broker e à propriedade de durabilidade discutida acima:

- O processo de **API** (`node src/api.js`, porta 3000) serve as rotas REST do Express, lê e escreve no SQLite local e atua como único produtor de eventos. Instancia o `RabbitMQEventPublisher`.
- O processo de **Worker** (`node src/worker.js`, porta 3001) é exclusivamente consumidor. Instancia o `RabbitMQEventConsumer` ligado à fila durável e o `WebSocketNotifier` que mantém o servidor WebSocket aberto para o app do prestador.

Os dois processos compartilham o código de domínio, de use cases e de repositórios, mas instanciam infraestrutura distinta no boot. A composição mora em `backend/src/composition.js`, em duas funções: `comporApi()` para o processo de API e `comporWorker()` para o de Worker.

Três adapters foram introduzidos na camada externa, todos implementando portas que vivem na camada de Use Cases:

- `RabbitMQEventPublisher` publica envelopes JSON no exchange `festalink.eventos` com routing key derivada do tipo do evento (`ReservaSolicitada` → `reserva.solicitada`). Usa channel em modo `confirm`, então cada publicação só resolve quando o broker confirma a persistência.
- `RabbitMQEventConsumer` declara fila durável e binding, consome com `prefetch(1)` e ACK manual, e despacha cada mensagem para uma função de callback configurada no `composition root`.
- `WebSocketNotifier` mantém um pool de conexões por `prestadorId` e expõe `notificarPrestadorDoSalao`, que consulta o repositório de salões e empurra o evento ao dono.

Os use cases `CriarReserva`, `AprovarReserva` e `RecusarReserva` recebem uma dependência `eventPublisher` e publicam o evento de domínio correspondente em microtarefa, depois do commit da transação. O wiring acontece em `composition.js` e respeita a regra de dependência: domínio e use cases não conhecem `amqplib` nem `ws`, só interfaces.

Demonstração da comunicação assíncrona

Foi escrito um cliente WebSocket de teste em `backend/scripts/ws-client.js`. Ele conecta no servidor WebSocket do Worker passando um JWT de prestador na query string e imprime no console todo evento recebido.

No teste manual, com a API e o Worker rodando em terminais separados e um prestador autenticado pelo cliente WS, foi disparado o fluxo:

1. Cliente envia `POST /reservas` via REST para a API.
2. A API persiste no SQLite, publica `ReservaSolicitada` no exchange RabbitMQ (com `publisher confirm`).
3. O RabbitMQ entrega a mensagem à fila `festalink.worker` por binding.
4. O Worker consome com ACK manual, despacha ao `WebSocketNotifier`.
5. O socket aberto pelo prestador recebe o envelope.
6. Em seguida, o prestador chama `PATCH /reservas/:id/aprovar`. O ciclo se repete com `ReservaAprovada`.

Os logs do Worker mostram cada passagem pelo consumer (`[consumer] ReservaSolicitada salao=4 reserva=4`), e o cliente WS registra a chegada dos dois eventos com timestamps abaixo de 50 ms de diferença entre a publicação e a entrega.

Para evidenciar a durabilidade, um segundo teste manual foi feito: o Worker foi derrubado com `Ctrl+C`, uma nova reserva foi criada via REST, a UI do RabbitMQ em `:15672` mostrou a mensagem acumulada na fila `festalink.worker`, e ao subir o Worker novamente o evento foi consumido e empurrado pelo WebSocket imediatamente, sem perda.

Suite de testes automatizados

Os testes da Sprint 2 verificam que `CriarReserva`, `AprovarReserva` e `RecusarReserva` chamam `eventPublisher.publicar` exatamente uma vez no caminho feliz, com o `type` e o `payload` corretos, e que não publicam nada quando a operação principal falha por autorização ou recurso inexistente. Os mocks usam apenas a interface (`publicar`), então sobreviveram à troca do adapter de Redis para RabbitMQ sem alteração. A regressão da Sprint 1 continua passando integralmente.

Limitações conhecidas e próximos passos

A publicação não é transacionalmente consistente com o INSERT — se a API morrer entre o commit no SQLite e o `publish` ao broker, o evento é perdido. Mesmo com o channel em modo `confirm`, isso só garante durabilidade depois que o broker confirma a recepção. Para a Sprint 2 esse risco é aceitável, dado o escopo acadêmico. Uma versão de produção usaria padrão Outbox: persistir o evento na mesma transação do SQLite e ter um publisher dedicado lendo a tabela e empurrando ao broker em segundo plano.

O WebSocket atual mantém o pool em memória do processo único do Worker. Em deploy escalado horizontalmente com múltiplos workers, RabbitMQ distribuiria as mensagens entre eles em modelo de competing consumers, e o socket de um prestador pode estar em outro worker que o que consome a mensagem. Para resolver isso seria necessário um fan-out por broker (exchange separado de notificações, com uma fila por worker) ou um registry compartilhado de sessões. O desenho atual com um Worker único cobre o escopo da disciplina.

A Sprint 3 adicionará o app cliente Flutter consumindo a REST com polling, conforme o item 3.4 do enunciado. A Sprint 4 adicionará o app prestador em Flutter que abrirá o WebSocket descrito aqui, completando o fluxo ponta-a-ponta.